M. Frisbie, *Building Browser Extensions*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_12

12. Permissions

Matt Frisbie[1]

(1) California, CA, USA

Browser extension permissions are conceptually identical to mobile app permissions. Both software platforms have the ability to access very powerful APIs, but to protect the end user permission to access these APIs must be explicitly requested on a piecewise basis. For both apps and extensions, permissions are declared by the developer in the codebase – for extensions, this is the extension manifest. In both cases, requesting access to trivial permissions will not require explicit user permission, but very powerful permissions will require the user to explicitly grant access. Furthermore, adding additional permissions in a subsequent update will typically require the user to explicitly accept this change in scope.

When building browser extensions, choosing permissions carefully is critically important. It has implications on how the marketplace listing will appear, the install and update flow for your extension, and how your extension will be reviewed by the marketplace.

Permissions Basics

To understand the basics of adding permissions, let's examine a very basic scenario that requires permissions. Begin with the following extension:

```
{
```

```
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "options_ui": {
    "open_in_tab": true,
    "page": "options.html"
  }
}
```

Example 12-1a manifest.json

```
<!DOCTYPE html>
<html>
  <body>
    <script src="options.js"></script>
  </body>
</html>
```

Example 12-1b options.html

```
chrome.storage.sync.set({ foo: "bar" });
```

Example 12-1c options.js

This very simple extension consists of an options page that, when opened, attempts to write a dummy value to the browser storage.

> **Note** This example uses an options page for two reasons: 1) because it is easier to access the console and reload the script, and 2) because when an error is thrown in the background page in the first turn of the event loop, the browser will mark the service worker as permanently idle and refuse to open the background worker developer tools.

After loading this extension and opening the options page, you will see this in the browser console (Figure 12-1).

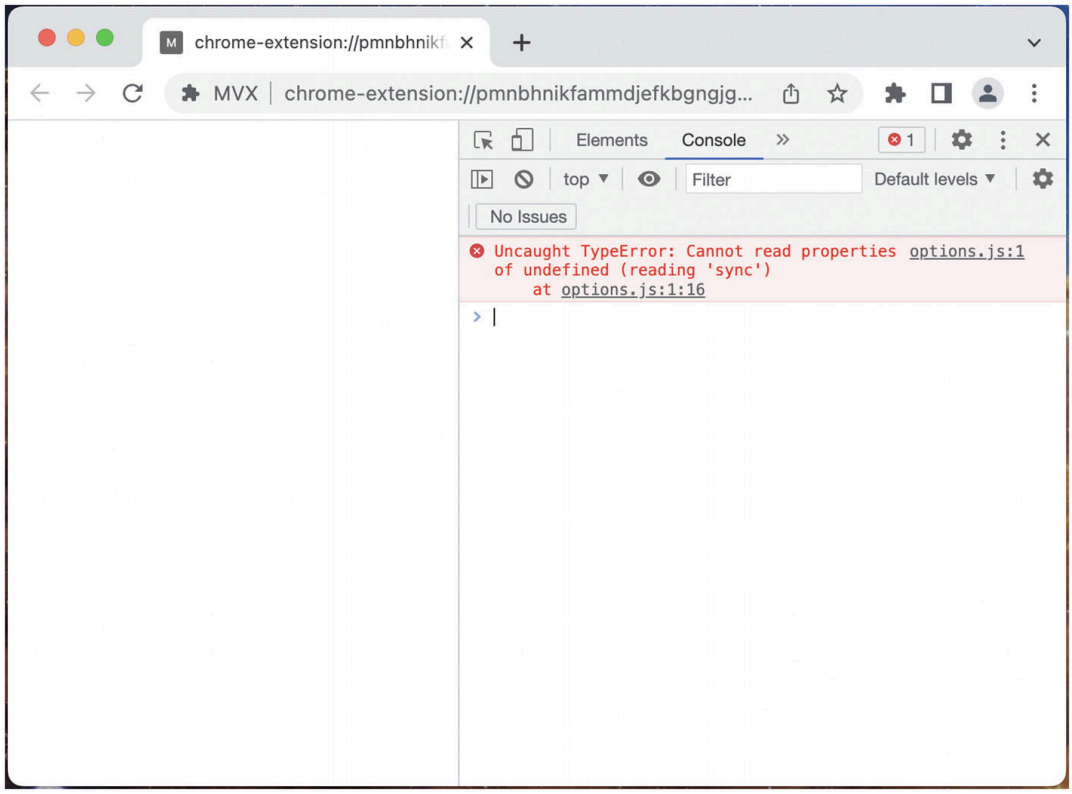

Figure 12-1 Error shown when attempting to set value in storage

The reason for this error is that the `chrome.storage` API requires the `storage` permission. Because the permission was not requested, the browser simply does not define the `storage` property on the `chrome` global object, and the attempted `sync()` call throws the above error.

This might surprise you, as the entire extension was loaded without issue. It is important to understand that the browser makes no assumptions about what permissions might be needed when loading the extension, or if a requested permission will be needed in the codebase at all. All permissions handling occurs at runtime.

To fix this example, all you must to is add the `storage` permission as shown below:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "options_ui": {
    "open_in_tab": true,
```

```
      "page": "options.html"
    },
    "permissions": ["storage"]
  }
```

Example 12-2 manifest.json with `storage` permissions added

After reloading the extension, the options page will no longer throw the error. This is because you have successfully added the needed permissions to use the `chrome.storage` API.

Checking Permissions

You can programmatically read all the permissions the current execution context has access to via the `chrome.permissions.getAll()` method. Update the `options.js` script from above to log the extension's permissions:

```
chrome.storage.sync.set({ foo: "bar" });
chrome.permissions.getAll().then(console.log);
```

Example 12-3 options.js

Reload the extension, and reload the options page to yield as in Figure 12-2.

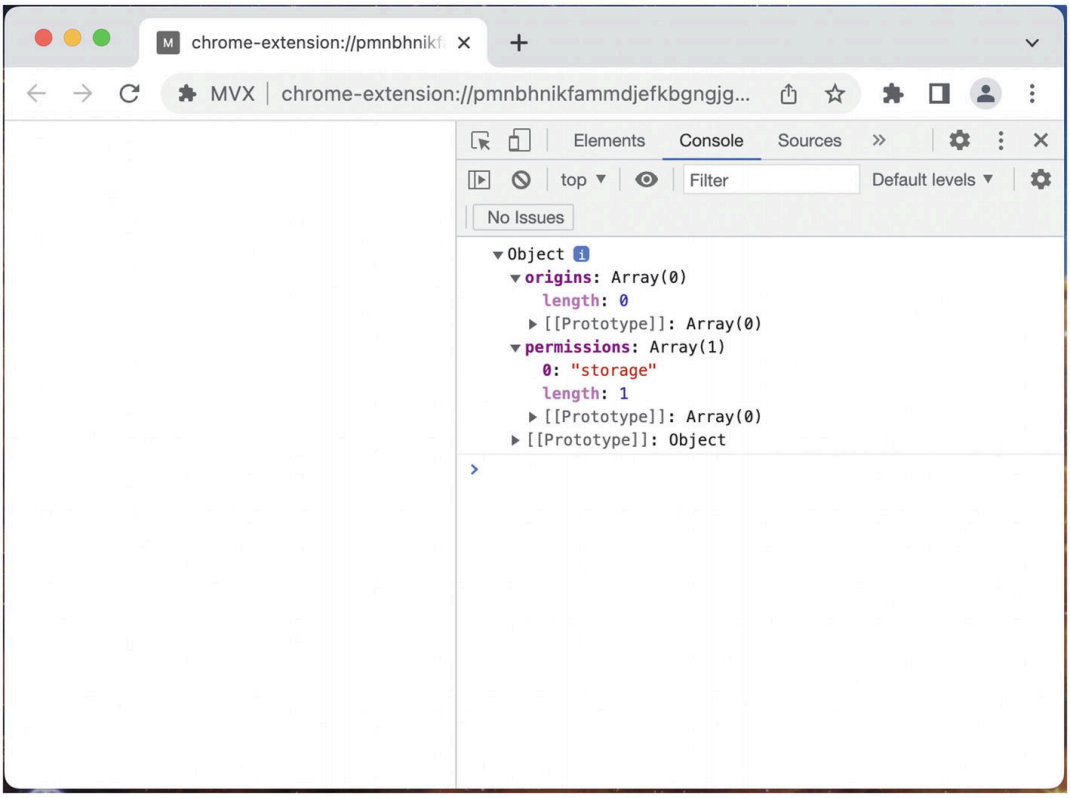

Figure 12-2 Logging the extension's current permissions

> **Note** Once granted, optional permissions (covered in upcoming sections) are no different than required permissions, so `chrome.permissions.getAll()` computes a merged list of all the required *and* optional permissions.

Using Optional Permissions

As the name indicates, optional permissions are permissions that can be optionally requested from the user. Optional permissions serve two primary use cases:

- Permissions that will be requested right when the extension first loads, but that are not critical for the extension to operate properly. This is analogous to a mobile app requesting access to your location right when you open it for the first time.
- Permissions that will be requested only after the user performs some action that triggers the request, such as clicking a "connect to foobar.com" button. This is analogous to a mobile app requesting

access to your device's photos only after you click a "share image" button.

Some examples of situations where optional permissions are useful

- You wish to allow users more control over the extension by allowing them to turn off some permissions, but still use some features of the extension
- You've already launched an extension but want to add more permissions without forcing the user to re-enable the extension
- You want to reduce the number of extension warnings that are displayed to the user after an install

Granting Permissions Declaratively vs. Imperatively

Whereas regular web browser permissions can be acquired *declaratively*, optional extension permissions must be acquired *imperatively*. Consider the following example of regular JavaScript running in a web page:

```
navigator.geolocation.getCurrentPosition(() => {})
```

The web browser geolocation API is not accessible by default, the user must grant access. However, the API is structured that simply calling this method will prompt the user with a dialog to grant permission; if the user grants permission, the script can proceed with execution as if the permission was granted all along.

Contrast this with extension example from the beginning of the chapter: calling a method without permissions will simply throw an error with no recourse. When using API methods requiring optional permissions, you must use the permissions API to check if the extension has permissions, request permissions if they are not granted, and only then can you proceed with the permissioned API call. This is demonstrated in the following example:

```
{
  "name": "MVX",
  "version": "0.0.1",
```

```
  "manifest_version": 3,
  "options_ui": {
    "open_in_tab": true,
    "page": "options.html"
  },
  "optional_permissions": ["storage"]
}
```

Example 12-4a manifest.json

```
<!DOCTYPE html>
<html>
  <body>
    <button id="save">Save</button>
    <script src="options.js"></script>
  </body>
</html>
```

Example 12-4b options.html

```
const permissions = {
  permissions: ["storage"],
};
document.querySelector("#save").addEventListener(
  "click",
  async () => {
    if (!(await chrome.permissions.contains(permissions))) {
      await chrome.permissions.request(permissions);
    }
    chrome.storage.sync.set({ foo: "bar" });
  });
```

Example 12-4c options.js

Permission Request Idempotence

The previous example can be simplified even further. Requesting permissions is idempotent, so you can simply call `request()` each time with no penalty:

```
document.querySelector("#save").addEventListener(
```

```
    "click",
    async () => {
      await chrome.permissions.request(permissions);
      chrome.storage.sync.set({ foo: "bar" });
    });
```

Host Permissions

Beginning in manifest v3, `permissions` and `host_permissions` were split out into separate fields, with a mirrored split between `optional_permissions` and `optional_host_permissions`. Conceptually, the difference is straightforward: `permissions` are the APIs you need access to, and `host_permissions` are the origins you need to use the APIs in. As discussed in the *Extension Manifest* chapter, the host permissions list effectively creates a whitelist of domains with extra privileges.

According to MDN, an origin matching the whitelist grants access to the following:

- `XMLHttpRequest` and `fetch` access to those origins without cross-origin restrictions (even for requests made from content scripts)
- The ability to read tab-specific metadata without the `tabs` permission, such as the `url`, `title`, and `favIconUrl` properties of `Tab` objects
- The ability to inject scripts programmatically using `tabs.executeScript()` into pages served from those origins
- The ability to access cookies for that host using the cookies API, as long as the `cookies` API permission is also included
- The ability to bypass tracking protection for extension pages where a host is specified as a full domain or with wildcards. Content scripts, however, can only bypass tracking protection for hosts specified with a full domain

The only difference between checking and requesting `optional_permissions` vs. `optional_host_permissions` is the use of the `origins` property instead of `permissions` property. For example, checking for `google.com` origins would be done in the following way:

```
chrome.permissions.contains({ origins: ["*://*.google.com"] })
```

Permissions Lifetime

Once a permission is granted, the extension has that permission for the entire lifetime of the extension. However, it is possible that the user can explicitly revoke that permission via the browser's extension management page. If the extension is uninstalled and then reinstalled, all the optional permissions that were formerly granted are lost.

Permissions Warnings

For permissions such as `alarms` and `storage`, the APIs they enable are of little consequence to the user and therefore they can be accessed silently. Conversely, some permissions are sufficiently powerful and therefore require the user to explicitly grant access to the extension. This takes the form of a dialog box with a description of what the permissions entail. For example, the following is what displays in Google Chrome when requesting the `tabs` permission (Figure **12-3**).

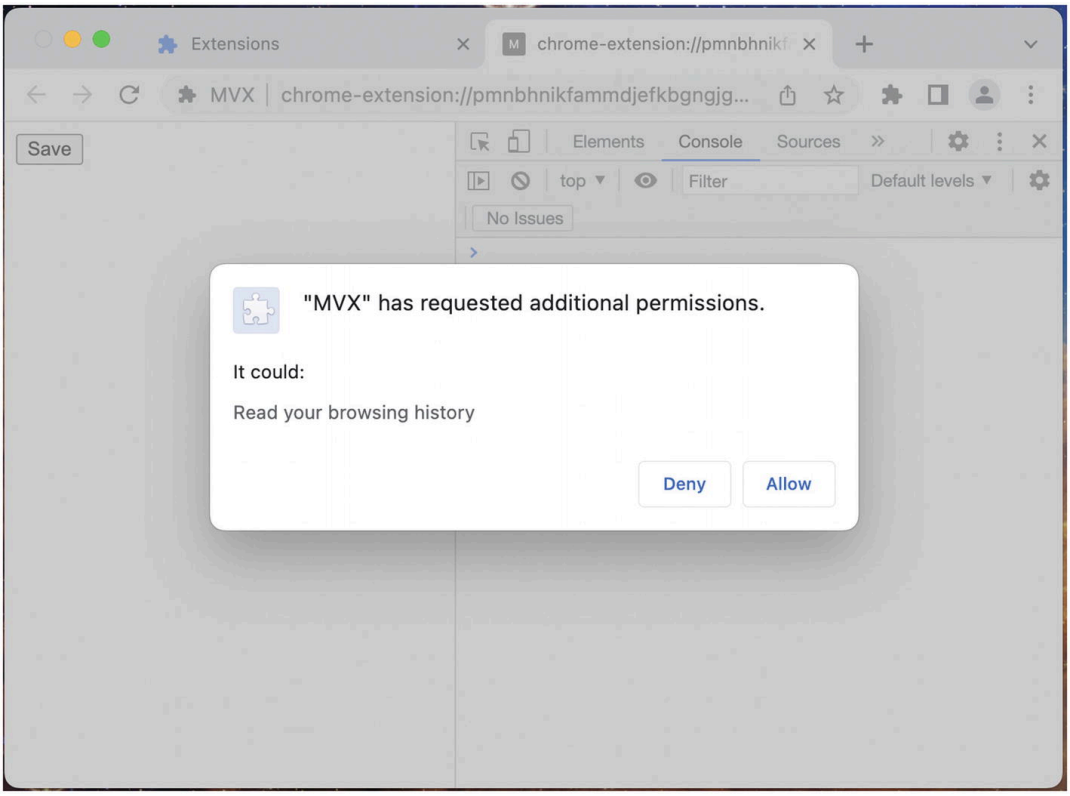

Figure 12-3 Prompt displayed after requesting `tabs` permission

For the `permissions` and `host_permissions` lists, the warning dialog will be displayed immediately after installing, and before the extension has an opportunity to execute any code. If the permission is denied, the extension will not be installed.

For `optional_permissions` and `optional_host_permissions`, the warning will only display when the extension makes the request for those permissions. If the permission is denied, the extension will continue to operate as it did before without the additional permissions.

> **Note** Unlike HTML5 permissions, the extension will be able to make another request for the optional permission if it is initially denied.

Testing Permissions Warnings

When developing your extension, you will notice that loading an unpacked will not show the permission warning dialog when initially installing the extension. (Optional permissions will still show the warning dialog.) Of course, this is to avoid annoying the developer when they repeatedly reinstall their unfinished extension.

manfest.json - Trivial one-file browser extension used to generate permission warning dialog

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "permissions": ["tabs"]
}
```

To see the install-time permission warning dialog, you will need to pack the extension into a `.crx` file and then load that into your browser.

- Click “Pack extension” from the extension management page (Figure 12-4)
- Select the extension folder (the same folder you would select when loading unpacked) (Figure 12-5)
- Leave the `.pem` field blank
- The browser will generate two files, a `.crx` and a `.pem`.
- Drag and drop the `.crx` file into the extension management page to install it. If you did this correctly, the browser will prompt you with the permission warning dialog. (Figures 12-6 and 12-7)
- Delete the two created files, they have no further use.

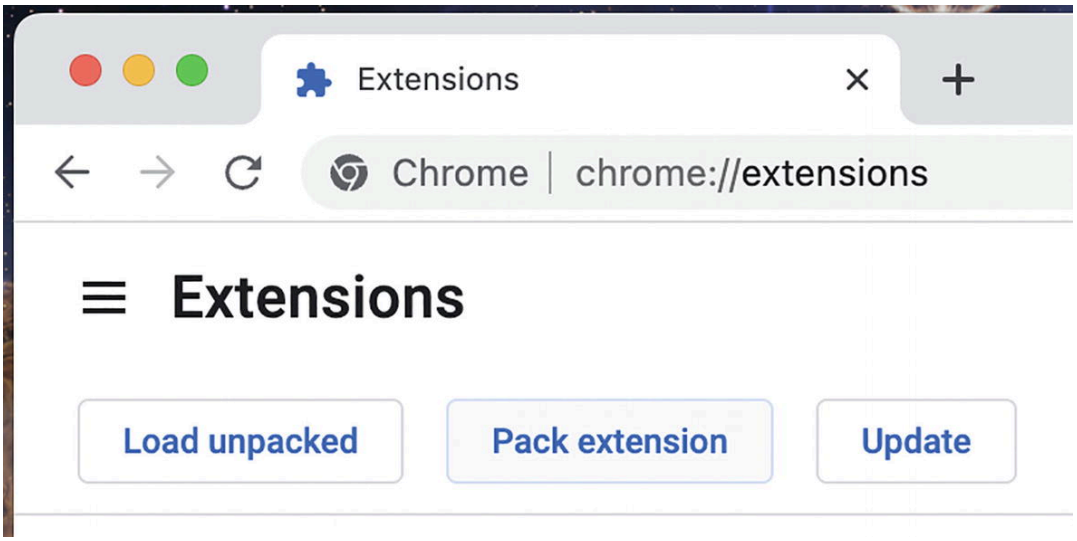

Figure 12-4 The “Pack extension” button on the extension management page

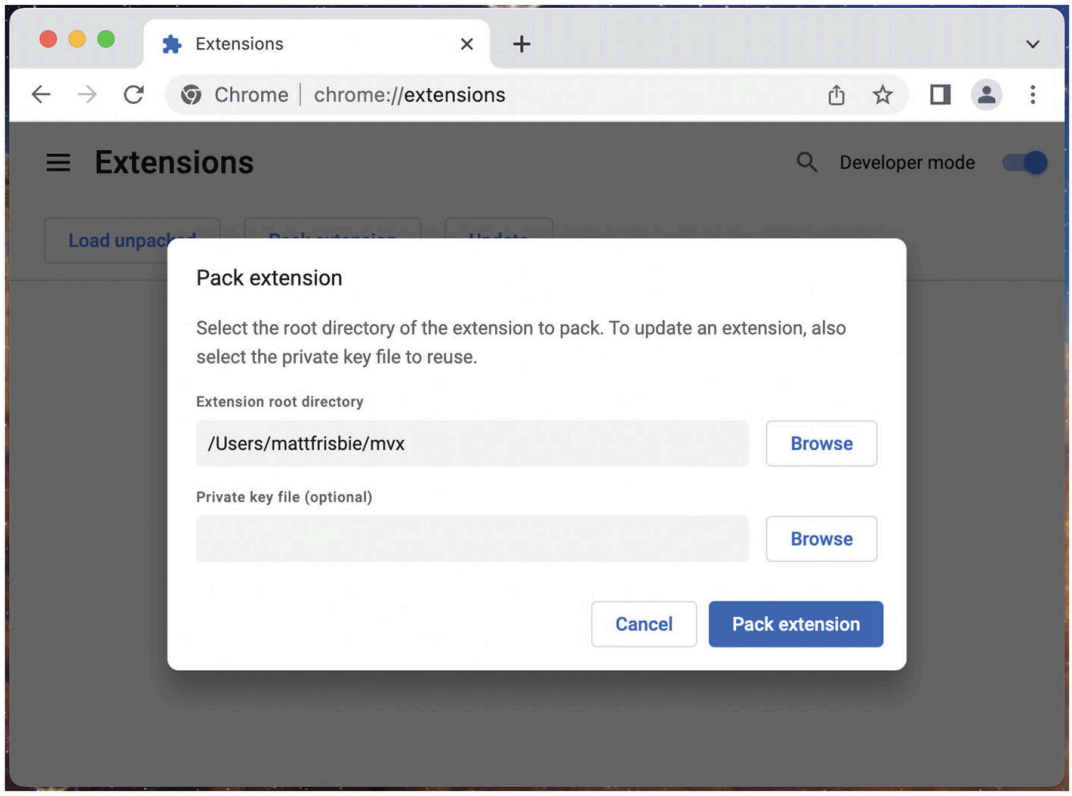

Figure 12-5 The “Pack extension“ dialog

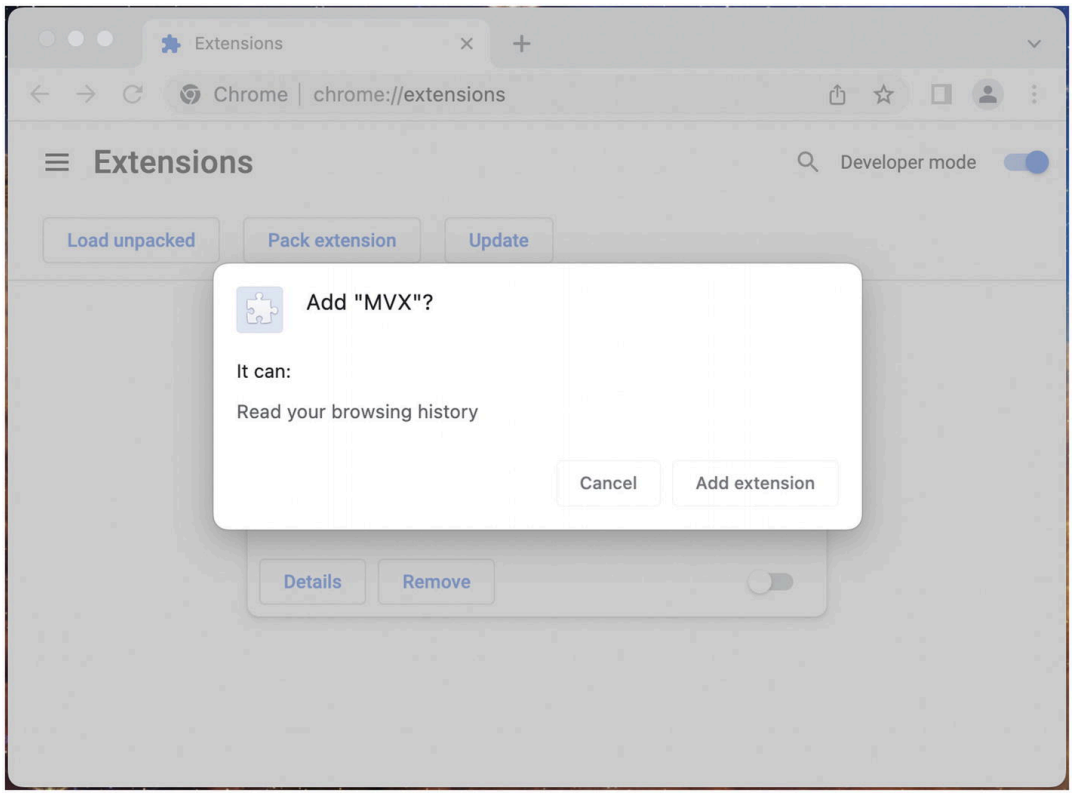

Figure 12-6 The install-time permissions warning dialog

NoteIt used to be possible to install and use an extension with a `.crx` file – this is no longer the case. It can still be used to test warnings as

shown here, but as you will see the extension will be permanently disabled by Chrome.

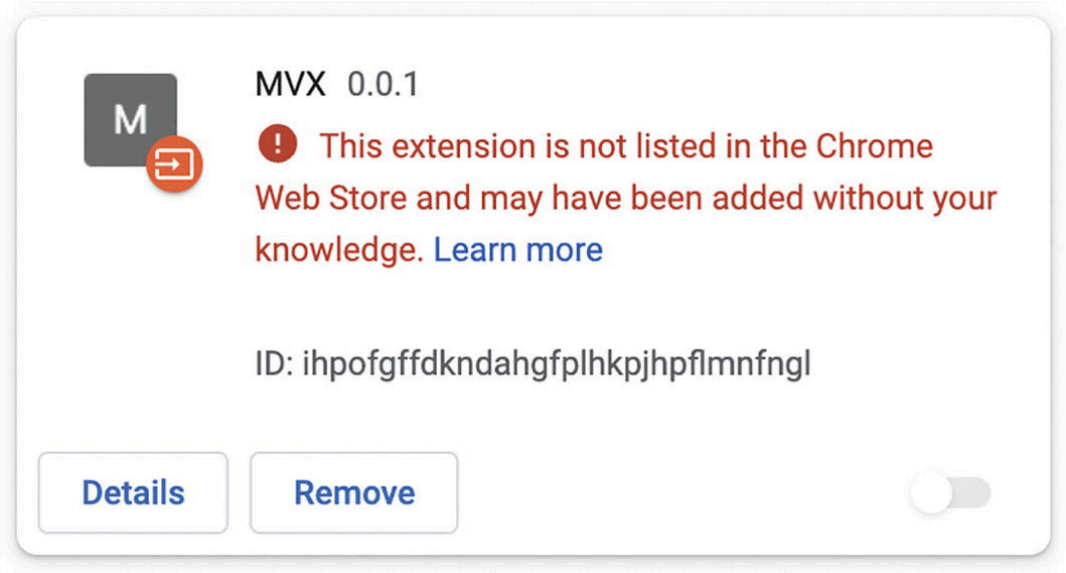

Figure 12-7 The disabled `.crx` extension

Considerations for Published Extensions

When publishing your extension to a marketplace, the permissions you select can have implications outside of your codebase.

Triggering the Slow Review Queue

When building an extension, you will probably not have any discretion for selecting which permissions are needed. After all, the needs of the extension are absolute, and the permissions needed to make it function will flow from there. However, if the speed of review for your extension is important, then it is important to understand that requiring some permissions such as `<all_urls>` will cause your extension to be placed in a slower review queue.

Tip I have found in my experience that this queue will increase your extension review time from less than 24 hours to a few days.

Auto-Disable Updates

One critically important consideration is what happens when required permissions that will show warnings are added to an extension in an update. The browser silently updates extensions in the background, so it won't show the warning dialog as soon as the update is installed. Instead, it will silently disable the extension and add a small notification to the browser toolbar, indicating action is required. To re-enable the extension, the user must open this notification and accept the new permissions (Figures 12-8 and 12-9). This flow is shown in the following.

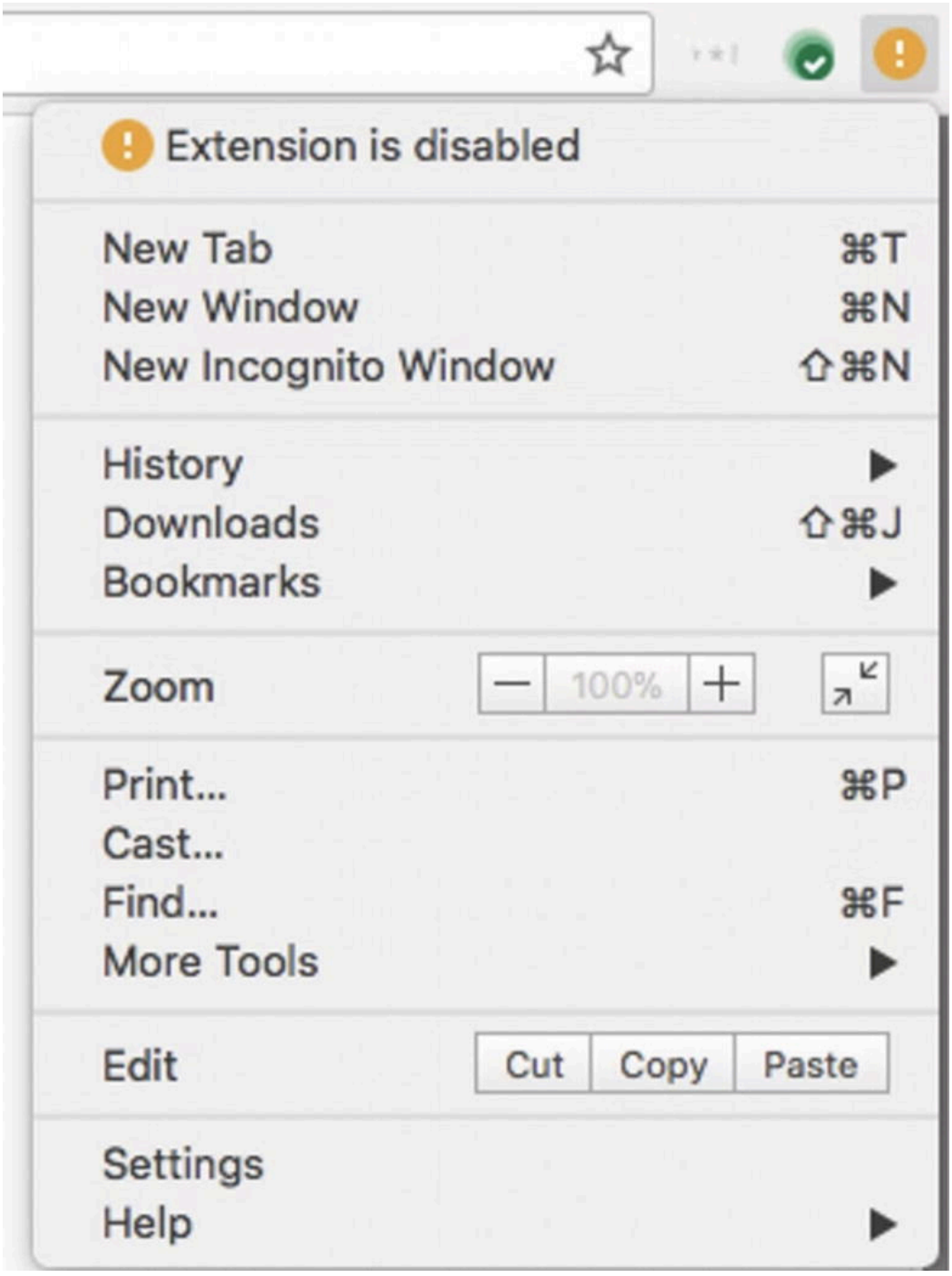

Figure 12-8 The disabled extension notification

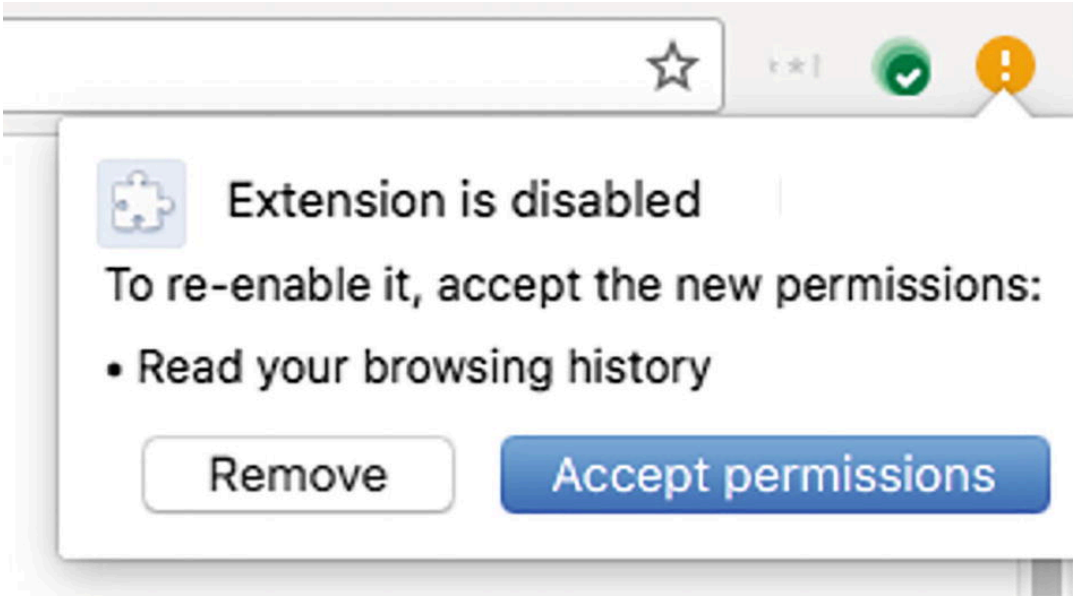

Figure 12-9 The permissions dialog to re-enable the extension

Permissions List

The following is a list of all permissions, what they enable, and the warning message they will incur (if any).

> **Note** This permissions list includes values from different browser vendors and different manifest versions. Some permissions may be deprecated.

> **Note** For consistency, the APIs are displayed inside the `chrome` namespace. This namespace is valid in all Chromium and Firefox browsers.

activeTab

The `activeTab` permission is commonly misunderstood – largely in part to its name. Extension developers were experiencing the following issues:

- Extensions only needed access to the "active tab," meaning the tab which was currently focused by the browser.
- Extensions needed access to the active tab irrespective of its domain, but did not want to incur the off-putting "Read and change all your data on the websites you visit" warning message.
- Extensions only needed temporary access to the active tab.

From this, the `activeTab` permission was born. This permission does not allow access to anything new: requesting the `<all_urls>` permission would make the `activeTab` permission pointless. However, by requesting `activeTab` instead of `<all_urls>`, the extension is granted extra permissions only after a limited set of user interactions without showing any warning to the user.

The allowed user interactions are:

- Executing an action
- Executing a context menu item
- Executing a keyboard shortcut command
- Accepting a suggestion from the Omnibox API

Once one of these user interactions occurs, the extension is temporarily granted the following extra permissions for that tab:

- Call `scripting.executeScript()` or `scripting.insertCSS()` on the active tab.
- Get the URL, title, and favicon for that tab via an API that returns a `tabs.Tab` object (essentially the same as the `tabs` permission).
- Intercept network requests in the tab to the tab's main frame origin using the `webRequest` API. The extension temporarily gets host permissions for the tab's main frame origin.

These extra permissions only last until the tab is navigated or is closed. This permission is available in Chromium browsers and Firefox.

alarms

- Gives your extension access to the `chrome.alarms` API
- Available for Chromium browsers and Firefox

background

- Makes Chrome start up early and shut down late, so that extensions can have a longer life
- Available for Chromium browsers and Firefox

When any installed extension has background permission, Chrome runs as a background process as soon as the user logs into their computer and before the user launches Chrome. The background permission also makes Chrome continue running after its last window is closed until the user explicitly quits Chrome.

The intended use of the `background` permission is to enable background scripts to perform useful work without being bound to the lifetime of the browser window.

bookmarks

- Grants your extension access to the `chrome.bookmarks` API
- Available for Chromium browsers and Firefox
- Chrome warning message: “Read and change your bookmarks”
- Firefox warning message: “Read and modify bookmarks”

browserSettings

- Enables an extension to modify certain global browser settings. Each property of this API is a `BrowserSetting` object, providing the ability to modify a particular setting
- Available for Firefox
- Firefox warning message: “Read and modify browser settings”

browsingData

- Gives your extension access to the `chrome.browsingData` API
- Available for Chromium browsers and Firefox

- Firefox warning message: “Clear recent browsing history, cookies, and related data”

captivePortal

- Determine the captive portal state of the user’s connection. A captive portal is a web page displayed when a user first connects to a Wi-Fi network
- Available for Firefox

certificateProvider

- Gives your extension access to the `chrome.certificateProvider` API
- Available for Chromium browsers

clipboardRead

- Required if the extension uses `document.execCommand('paste')`
- Available for Chromium browsers and Firefox
- Chrome warning message: “Read data you copy and paste”
- Firefox warning message: “Get data from the clipboard”

clipboardWrite

- Indicates the extension uses `document.execCommand('copy')` or `document.execCommand('cut')`
- Available for Chromium browsers and Firefox
- Chrome warning message: “Modify data you copy and paste”
- Firefox warning message: “Input data to the clipboard”

contentSettings

- Grants your extension access to the `chrome.contentSettings` API.

- Available for Chromium browsers and Firefox
- Chrome warning message: “Change your settings that control websites’ access to features such as cookies, JavaScript, plugins, geolocation, microphone, camera etc.”

contextMenus

- Gives your extension access to the `chrome.contextMenus` API
- Available for Chromium browsers and Firefox

contextualIdentities

- List, create, remove, and update contextual identities, more commonly known as “containers”
- Available for Firefox

cookies

- Gives your extension access to the `chrome.cookies` API
- Available for Chromium browsers and Firefox

debugger

- Grants your extension access to the `chrome.debugger` API
- Available for Chromium browsers and Firefox
- Multiple Chrome warning messages:
  - “Access the page debugger backend”
  - “Read and change all your data on the websites you visit”

declarativeContent

- Gives your extension access to the `chrome.declarativeContent` API
- Available for Chromium browsers

declarativeNetRequest

- Grants your extension access to the `chrome.declarativeNetRequest` API
- Available for Chromium browsers
- Chrome warning message: “Block page content”

declarativeNetRequestFeedback

- Grants the extension access to events and methods within the `chrome.declarativeNetRequest` API which return information on declarative rules matched
- Available for Chromium browsers

declarativeWebRequest

- Gives your extension access to the `chrome.declarativeWebRequest` API
- Available for Chromium browsers

desktopCapture

- Grants your extension access to the `chrome.desktopCapture` API
- Available for Chromium browsers
- Chrome warning message: “Capture content of your screen”

Devtools page

- Technically not a permission, this is an extra warning that displays in Firefox when `devtools_page` is used in the manifest
- Firefox warning message: “Extend developer tools to access your data in open tabs”

dns

- Enables an extension to resolve domain names using the `dns` API
- Available for Firefox

documentScan

- Gives your extension access to the `chrome.documentScan` API
- Available for Chromium browsers

downloads

- Grants your extension access to the `chrome.downloads` API
- Available for Chromium browsers and Firefox
- Chrome warning message: “Manage your downloads”
- Firefox warning message: “Download files and read and modify the browser’s download history”

downloads.open

- Grants your extension access to only the `chrome.downloads.open` API
- Chrome warning message: “Open downloaded files”
- Firefox warning message: “Open files downloaded to your computer”

enterprise.deviceAttributes

- Gives your extension access to the `chrome.enterprise.deviceAttributes` API
- Available for Chromium browsers

enterprise.hardwarePlatform

- Gives your extension access to the `chrome.enterprise.hardwarePlatform` API

- Available for Chromium browsers

enterprise.networkingAttributes

- Gives your extension access to the `chrome.enterprise.networkingAttributes` API
- Available for Chromium browsers

enterprise.platformKeys

- Gives your extension access to the `chrome.enterprise.platformKeys` API
- Available for Chromium browsers

experimental

- Required if the extension uses any `chrome.experimental.*` APIs
- Available for Chromium browsers

fileBrowserHandler

- Gives your extension access to the `chrome.fileBrowserHandler` API
- Available for Chromium browsers

fileSystemProvider

- Gives your extension access to the `chrome.fileSystemProvider` API
- Available for Chromium browsers

find

- Enable the extension to find text in a web page and highlight matches with the `find` API
- Available for Firefox
- Firefox warning message: “Read the text of all open tabs”

fontSettings

- Gives your extension access to the `chrome.fontSettings` API
- Available for Chromium browsers

gcm

- Gives your extension access to the `chrome.gcm` API
- Available for Chromium browsers

geolocation

- Allows the extension to use the HTML5 geolocation API without prompting the user for permission
- Available for Chromium browsers and Firefox
- Chrome warning message: “Detect your physical location”
- Firefox warning message: “Access your location”

history

- Grants your extension access to the `chrome.history` API
- Available for Chromium browsers and Firefox
- Chrome warning message: “Read and change your browsing history”
- Firefox warning message: “Access browsing history”

Host Permissions

Host permissions can either be requested globally, or for only a subset of hosts. These permissions can be requested on all browsers, and the warning message displayed may appear in several different ways.

> **Note** It may be possible to avoid declaring any host permissions by using the `activeTab` permission.

Global Host Permission

Global host permission means the extension has access to every host with a permitted protocol. This is appropriate for extensions that need to manage all the browser's pages and traffic all the time. There are several different ways to request this, but they all have the same effect:

- `http://*/*`
- `https://*/*`
- `*://*/*`
- `<all_urls>`

This will show the following warning messages:

- Chrome warning message: "Read and change all your data on the websites you visit"
- Firefox warning message: "Access your data for all websites"

> **Tip** If your extension uses this permission, your extension will be placed in the manual review queue when submitting to extension marketplaces. As a result, it will take significantly longer to be approved.

Partial Host Permission

Partial host permission means the extension has access to only hosts that match one or more matchers. This is appropriate for extensions that know ahead of time the set of hosts that it will need to manage or communicate with.

> **Note** Refer to the Extension Manifest chapter for coverage on how matchers work.

The warning displayed will vary based on how many hosts are covered. A simple single host matcher for `foobar.com` will display the following warnings:

- Chrome warning message: "Read and change your data on `foobar.com`"
- Firefox warning message: "Grants the extension access to **`https://foobar.com`**"

If more than one host is covered, the browser will adjust the warning message based on how many hosts are covered. For example, Firefox also will show the following messages:

- Access your data for sites in the [named] domain
- Access your data in # other domains
- Access your data for [named site]
- Access your data on # other sites

identity

- Gives your extension access to the `chrome.identity` API
- Available in Chromium browsers and Firefox

idle

- Gives your extension access to the `chrome.idle` API
- Available for Chromium browsers and Firefox

loginState

- Gives your extension access to the `chrome.loginState` API
- Available in Chromium browsers

management

- Grants the extension access to the `chrome.management` API
- Available for Chromium browsers and Firefox
- Chrome warning message: “Manage your apps, extensions, and themes”
- Firefox warning message: “Monitor extension usage and manage themes”

menus

- Add items to the browser’s menu system with the `menus` API
- Available for Firefox

> **Note** This API is modeled on Chrome's `contextMenus` API, which enables Chrome extensions to add items to the browser's context menu. The `menus` API adds a few features to Chrome's API.

menus.overrideContext

- Hide all default Firefox menu items in favor of providing a custom context menu UI
- Available for Firefox

nativeMessaging

- Gives the extension access to the native messaging API
- Available for Chromium browsers and Firefox
- Chrome warning message: “Communicate with cooperating native applications”
- Firefox warning message: “Exchange messages with programs other than Firefox”

notifications

- Grants your extension access to the `chrome.notifications` API
- Available for Chromium browsers and Firefox
- Chrome warning message: “Display notifications”
- Firefox warning message: “Display notifications to you”

pageCapture

- Grants the extension access to the `chrome.pageCapture` API
- Available for Chromium browsers and Firefox
- Chrome warning message: “Read and change all your data on the websites you visit”

pkcs11

- The pkcs11 API enables an extension to enumerate PKCS #11 security modules and to make them accessible to the browser as sources of keys and certificates
- Available for Firefox
- Firefox warning message: “Provide cryptographic authentication services”

  Per Wikipedia:

- *Most commercial certificate authority (CA) software uses PKCS #11 to access the CA signing key or to enroll user certificates. Cross-platform software that needs to use smart cards uses PKCS #11, such as Mozilla Firefox and OpenSSL (using an extension).*

platformKeys

- Gives your extension access to the `chrome.platformKeys` API
- Available in Chromium browsers

power

- Gives your extension access to the `chrome.power` API
- Available for Chromium browsers

printerProvider

- Gives your extension access to the `chrome.printerProvider` API
- Available for Chromium browsers

printing

- Gives your extension access to the `chrome.printing` API
- Available for Chromium browsers

printingMetrics

- Gives your extension access to the `chrome.printingMetrics` API
- Available for Chromium browsers

privacy

- Gives the extension access to the `chrome.privacy` API
- Available for Chromium browsers and Firefox
- Chrome warning message: “Change your privacy-related settings”
- Firefox warning message: “Read and modify privacy settings”

processes

- Gives your extension access to the `chrome.processes` API
- Available for Chromium browsers

proxy

- Grants the extension access to the `chrome.proxy` API
- Available for Chromium browsers and Firefox

- Chrome warning message: “Read and change all your data on the websites you visit”
- Firefox warning message: “Control browser proxy settings”

scripting

- Gives your extension access to the `chrome.scripting` API
- Available for Chromium browsers and Firefox

search

- Gives your extension access to the `chrome.search` API
- Available for Chromium browsers and Firefox

sessions

- Gives your extension access to the `chrome.sessions` API
- Available for Chromium browsers and Firefox
- Firefox warning message: “Access recently closed tabs”

signedInDevices

- Gives your extension access to the `chrome.signedInDevices` API
- Available for Chromium browsers

storage

- Gives your extension access to the `chrome.storage` API
- Available for Chromium browsers and Firefox

system.cpu

- Gives your extension access to the `chrome.system.cpu` API
- Available for Chromium browsers

system.display

- Gives your extension access to the `chrome.system.display` API
- Available for Chromium browsers

system.memory

- Gives your extension access to the `chrome.system.memory` API
- Available for Chromium browsers

system.storage

- Grants the extension access to the `chrome.system.storage` API
- Available for Chromium browsers
- Chrome warning message: “Identify and eject storage devices”

tabCapture

- Grants the extensions access to the `chrome.tabCapture` API
- Available for Chromium browsers
- Chrome warning message: “Read and change all your data on the websites you visit”

tabGroups

- Gives your extension access to the `chrome.tabGroups` API
- Available for Chromium browsers

tabHide

- Allows the extension to use the `tabs.hide()` API
- Available for Firefox

tabs

- Grants the extension access to privileged fields of the Tab objects used by several APIs including `chrome.tabs` and `chrome.windows`
- Available for Chromium browsers and Firefox
- Chrome warning message: “Read your browsing history”
- Firefox warning message: “Access browser tabs”

> **Note** In many circumstances the extension will not need to declare the `tabs` permission to make use of these APIs.

theme

- Enables browser extensions to update the browser theme
- Available for Firefox

topSites

- Grants the extension access to the `chrome.topSites` API
- Available for Chromium browsers and Firefox
- Chrome warning message: “Read a list of your most frequently visited websites”
- Firefox warning message: “Access browsing history”

tts

- Gives your extension access to the `chrome.tts` API
- Available for Chromium browsers

ttsEngine

- Grants the extension access to the `chrome.ttsEngine` API
- Available for Chromium browsers
- Chrome warning message: “Read all text spoken using synthesized speech”

unlimitedStorage

- Provides an unlimited quota for storing client-side data, such as databases and local storage files. Without this permission, the extension is limited to 5 MB of local storage
- Available for Chromium browsers and Firefox
- Firefox warning message: “Store unlimited amount of client-side data”

> **Note** This permission applies only to Web SQL Database and application cache. Also, it doesn't currently work with wildcard subdomains such as http://*.example.com.

vpnProvider

- Gives your extension access to the `chrome.vpnProvider` API
- Available for Chromium browsers

wallpaper

- Gives your extension access to the `chrome.wallpaper` API
- Available for Chromium browsers

webNavigation

- Grants the extension access to the `chrome.webNavigation` API
- Available for Chromium browsers and Firefox
- Chrome warning message: “Read your browsing history”
- Firefox warning message: “Access browser activity during navigation”

webRequest

- Gives your extension access to the `chrome.webRequest` API
- Available in Chromium browsers and Firefox

webRequestBlocking

- Required if the extension uses the `chrome.webRequest` API in a blocking fashion
- Available in Chromium browsers and Firefox

Summary

In this chapter, you learned about what permissions are, what they do, and how to declare them. The chapter also covered the difference between required, optional, and host permissions. It also explained all the tricky implications with selecting and changing permissions.

In the next chapter, we will cover all the different aspects of how extensions can use and modify the browser's network capabilities, as well as how it can creatively use content scripts to act as an agent of the authenticated user. This chapter also covers the important differences between `webRequest` and `declarativeNetRequest`.